

BLM 1012

Yap. Prog. Giriş

instructions

Algorithms are set of certain, sequential and finite

Algorithmic Operations

→ Sequential: assignment, math, comparison
: $+$, $-$, $<$, $>$, ...

→ Conditional: if

→ Repetitive: For, while

Algorithms

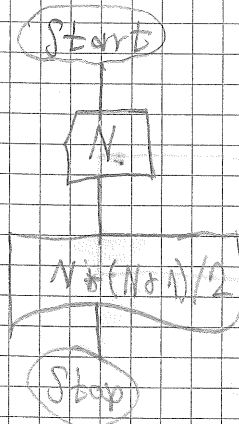
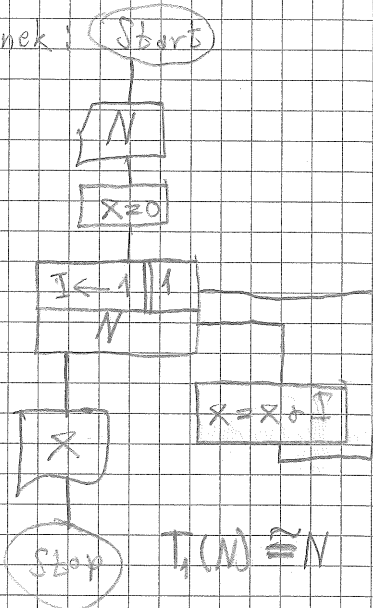
efficient & economical

(time) (space)

→ Accuracy & precision, resource utilization, CPU, memory, readability

almost
trade-off always happens

Örnek: Start



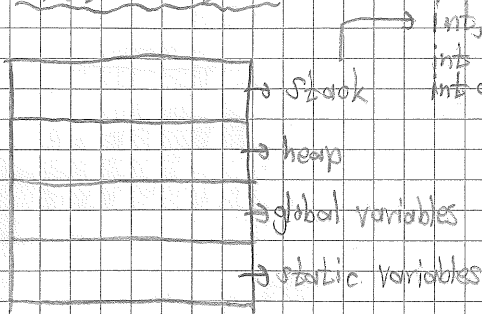
$$T_2(N) = 2$$

Pointers & Memory Organisation

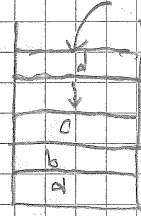
Bellekte sana yer ayrılır. Kod, veriler, bilgiler var.

* Sana gelişigüzel yer vermez. Bir alan ayırır.

Data Section



int, float
int a[10];
int *a;



Stack yapısı

Pointer Variables

symali. $\text{int } a;$
 $\text{int } *pa;$ $\rightarrow pa = \&a; / pa = \&a[0];$

değer değil, int tipinde
bir değişkenin adresini tutar.

* Pointer ve adresi %p ile yazılır.

* Pointerin bulunduğu değer için
dereferans yap. *pa diye yazılır.

%d / %f ile yaz.

* İçerik değiştirmek için adrese ihtiyaç var.

Address/Reference: &
Value/Dereference: *

Functions

type my func (parameters)
return

* Tekrar kullanılabilirlik sağlar.

void yazdır (int a[], int n){

* Başka bir fonksi-
yon içinde çağırabilirsin.



* Bellekte fonksiyonların kendi alanları var. İkinde de a, b, c'den tanımlayabilişin.

* Fonksiyonlar arası iş yaparken pointer'lar işine yarayacak.

Örni: Parametre olarak adres gönderirsen o adresle yapılan şey kalıcı olur.

$\frac{2}{k}$	3	4	$\frac{5}{k}$	$\frac{7}{b}$	$\frac{8}{1}$	9	$\frac{10}{b}$	$\frac{k}{0}$	$\frac{0}{3}$	$\frac{b}{7}$
			$\frac{k}{0}$	$\frac{b}{7}$						
$A[0] - A[k]$				$0 - k$				$0 - 3$		
3				3				$0 = (k+b)/2$		

2. tür $\leftarrow b=0, 0=(k+b)/2$ - Ardışık dizide eksik

3. tür $\leftarrow b=0, 0=(k+b)/2$ elemanı bulduk.

* C'de fonksiyonlar da çatırılan kullanılır. kopya oluşup kullanılır.

① Dizi almada shallow; değişkenlerde deep copy yapılır. sadece pointer'ına bakılır, yeni kopya yer açılır.

② Dizi return edilemez.

* X alıp iş yaparsan main'deki x hala aynı.

Dizi alıp iş yaparsan asıl diziyi değiştirir.

③ Pointer olabilir veya ④ çatırılan fonksiyon bitince stack'ten döndürebilir; silinir, değişkenler kaybolur.

⑤ Fonksiyonda parametreyi * ile tanımlıyorsun o bir adres olur. Çoğurken & ile değişkeni yaz.

⑥ Pointerlarla "string" dönebilirsin.

6. hafta

→ Matematiksel ifadeleri işledik. (Algoritma Analizi)

→ Resimler

• Bilgisayarda gerçek rastgele değil, sözde rastgelelik var.

Sayılar hep bir öncekinden üretilir. R[0] biliniyor ilinebilir.

Gözümü. R[0]'ı hep değiştir.

Ne kadar büyük olursa gerçek rastgeleliğe o kadar yakın olur.

• Zor aralıkm.

• Kapsalı aralıkta sayı üretilim. $\text{rand \% (Y-X)} + X$
[X, Y]

• 52 kartı dağıttım. ① Rand üret, bassa kartı ver.

52 yd) var. ② 1-52 arası → Rastgele iki sayı üretilip onları dizi yapı. Sırala. Çok kez yapı.

• Gaussian dağılım üret. Sıralı ikililerle yaptık.

• Rastgele yazıyor. Yönlere 0,1, -1 ile ayarla farklı sor-

① 18'li ② Adım matrisi yapı. tirlarda. Bir satırı x ve y'ye uygula.

• Π serisi:

$$\text{say} = \Pi r^2 = \Pi$$

$$N = 2.2$$

$$T = \frac{4 \cdot \text{say}}{4} = \text{say}$$

Monte Carlo
Simülasyonu



Stack'te main ve fonksiyonlar:

main'de 1. satıra gelinir. Orada f_1 'e girilir. f_1 'den devam
 f_1 'de 10. satırda f_2 'ye gir. f_2 11'den devam
 f_2 'de 30. satırda f_3 'e gir. f_3 31'den devam
 f_3 yapılır.
push
pop
İki biten main'den silinir.

→ Sonra main 3'de f_n görülür...

Recursive (Özyinelemeli) Fonksiyonlar

Fonksiyon kendi kendini çağırır.

Direkt $f \rightarrow f$

Dolaylı $f \rightarrow g, g \rightarrow f$

void $f2(\sim);$

if ($\sim$) return; // base case → ne olursa çıkarsın.

$f2(\sim);$ // recursive case

→ Recursive'de takip zor. Stack kullanabilirsin çizip.

Yaptık: Faktöriyel, 'is alma' → $x \cdot x^{n-1}$
→ $x^{n/2} \cdot x^{n/2} = \dots$ (sonra)

→ Linear Sort with Counting

→ Counting Sorts

$$\rightarrow \sum_{i=1}^N 1 = N$$

$$\sum_{i=1}^N \sum_{j=1}^N 1 = N^2$$

$$\sum_{i=1}^N \sum_{j=1}^N i = N^2 \frac{(N+1)}{2}$$

$$\sum_{i=1}^N \sum_{j=1}^N j = \sum_{i=1}^N \frac{N^2 (N+1)}{2}$$

$$\sum_{i=1}^N \sum_{j=1}^{i^2} 1 = \sum_{i=1}^N i^2 = \frac{N(N+1)(2N+1)}{6}$$

$$\sum_{i=1}^N i^3 = \left(\frac{N(N+1)}{2} \right)^2 \quad \sum_{i=1}^N \frac{i^2(i+1)}{2} = \sum_{i=1}^N \frac{i^3 + i^2}{2}$$

$$\sum_{i=1}^{\log_2 N + 1} 1 = \log_2 N + 1 \quad T \geq T+1 \text{ olsa} \Rightarrow T = \sum_{i=0}^{\log_2 N} 2^i$$

$$* T = \sum_{i=0}^N A^i = \frac{A^{N+1} - 1}{A - 1} \quad * \sum_{i=0}^{\log_2 N} 2^i = \frac{2^{(\log_2 N + 1)} - 1}{2 - 1} = 2N - 1$$

$$* K = 2^N \Rightarrow \sum_{i=1}^N \sum_{j=1}^{2^N} 1 = \sum_{i=1}^N 2^N = N \cdot 2^N$$

$$\sum_{i=1}^N \sum_{j=1}^{2^N} i = \sum_{i=1}^N i \cdot 2^N = 2^N \cdot \frac{N(N+1)}{2}$$

$$* K = 2^N \quad \sum_{i=1}^N \sum_{j=1}^{\log_2 K} 1 = N^2 \quad * \sum_{i=1}^N \sum_{j=1}^i \left(\sum_{k=1}^j 1 \right) = \sum_{i=1}^N \sum_{j=1}^i \frac{j(j+1)}{2} = \frac{N(N+1)(N+2)}{6}$$

→ Fonksiyonlar

Çarpım tabiri ve terimler

→ Rekürsiften devam

• Rekürsif Fibonacci: $T(n) = T(n-1) + T(n-2) + 1$

↳ Kodu yaz, bağıntıyı yaz, denklemi çöz.

①

②

③

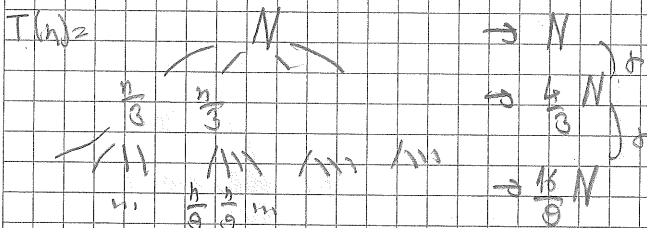
* $T(n) = 4 \cdot T\left(\frac{n}{3}\right) + 1$ → Her seriyedeki işlem yükü

1 Her seriyenin dal sayısı 2 Ağacın derinleşme miktarı → $h = \log_3 n$

$$\sum_{i=0}^h \frac{4^i}{1} \cdot 1 = \sum_{i=0}^{\log_3 n} 4^i = \frac{4^{\log_3 n + 1} - 1}{4 - 1} = \frac{n^{\log_3 4} - 1}{3}$$

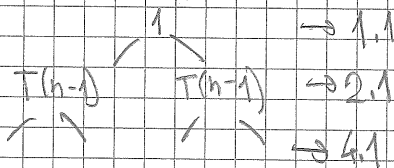
• $T(n) = 9T\left(\frac{n}{3}\right) + 1$

• $T(n) = 4T\left(\frac{n}{3}\right) + n$



$$N \cdot \sum_{i=0}^h \left(\frac{4}{3}\right)^i = \frac{\left(\frac{4}{3}\right)^{\log_3 n + 1} - 1}{\frac{4}{3} - 1} = \dots = O(n^{\log_3 4})$$

• $T(n) = 2T(n-1) + 1$



$$T(n) = \sum_{i=0}^n 2^i = \frac{2^{n+1} - 1}{2 - 1}$$

$\approx 2^n$ $K^n \neq n^n$

Fibonacci

$$T(n) = T(n-1) + T(n-2) = O\left(\frac{1+\sqrt{5}}{2}\right)^n$$

$$= (1.6)^n \approx 2^n / \text{kötü}$$

Fibonacci 2

$$F = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} f(n) & f(n) \\ f(n) & f(n-1) \end{bmatrix} = f^n$$

$$F_2 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

$$F_3 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 2 & 1 \end{bmatrix} \rightarrow \text{Matrisle } O(n) \text{ olur,}$$

Matris üssünü

$x^n = x^{n/2} \cdot x^{n/2}$ ile bulabiliriz.

Flood Fill Algorithm

4 yöne
bakmak

$$T(m, n) = 4(m-1, n-1) + 1 \quad \text{Rekürsifdir.}$$

2 boyutlu,
2 değişken var

- Her noktanın 4 yanına bak.
- Maliyetlidir.

```
void flood(x, y, new_col, old_col) {
    if (getpixel(x, y) == old_col) {
        putpixel(x, y, new_col)
        flood(x+1, y, ...)
        flood(x-1, y, ...)
        flood(x, y+1, ...)
        flood(x, y-1, ...)
    }
}
```

Dunwall

Din dash Riddle

Freeport drinks wine



g, w, b

rum

~~Lady Winslow~~

Doctor Marcollo beer

~~Chinese Canteen~~

Bird Maxium / Ztsin

Baroness Finch wine

green hat

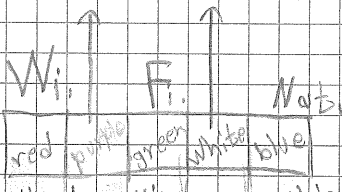
purple jockes

diamond

M, C, N

• White sits left of blue

spills her rum Ma Baletan



absinthe beer wine rum whis

Freeport

Dia. tin

bird

• From Freeport → red ✓

sits next to diamond ✓

• Baletan next to one with War Medal ✓

neighbor

drinks absinthe

→ whiskey beer

Wi ——— Freeport

Co ——— Dunwall

~~Ma~~ ——— ~~Dahokva (tin)~~

~~Na~~ ——— ~~Baletan (dia)~~

~~Pi~~ ——— ~~Karnacod~~

Det. Bulma

$$10 - \frac{13}{2} = 3.5$$

$$+2 \begin{bmatrix} -1 & 2 & 3 \\ 1 & 0 & 2 \\ 4 & 5 & -2 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 2 & 3 \\ 0 & 4 & 6 \\ 0 & 13 & 10 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 2 & 3 \\ 0 & 4 & 6 \\ 0 & 0 & -5 \end{bmatrix}$$

D. Projesi

$$\frac{13}{4} = 3.25$$

V. notu

turn 3 alinyon.

moves	1	0	0	3
	4	4	2	3
turn =	1	1	2	2

Scores

- ① O.s. T.p. Max
- ②



...../...../.....

purple

Table

W

Winslow
Marcello
Cortez
Notorio
Finch

Who? Winslow, Marcello, Finch, Cortez, Notorio

color red purple green white blue

red green
purple white blue

drink abs. beer wine rum whis

beer rum wine
whis, abs.

origin Freeport Boleton Dabokva Dunnell Korn

Dunnell
Carnacao

heirloom diamond war medal bird

Boleton
Dabokva (tin)
Freeport



Origins:

Wi | Fi | Ma | Na | Co
Freeport | | | | Dunnell

Dabokva
Boleton
Karnaca

Elbir doktora konusu → müzik tavsiyesi

Connected Component Labeling

For'la tüm haritayı gez. Nokta daaysa recursive bir "etiketle" ile nesneyi çiz. • Başsa dokunma, etiketle içinde sınırlardan çıkılırsa veya > 0 ise dokunma.

* 8 komşu için etiketle içinde görselleri de etiketle.

Kısaltma: Yön matrisi tut. For'la 4/8'e kadar etiket ($x + dx[yön]$, $y + dy[yön]$) -) girer.

→ longest sequence ; → concat: kombinasyon bulma
1) brute 2) sorted ; recursive sona ekleye eklege

tekrarlı → rec. içinde for, içinde rec.

tekrarsız → rec içindeki for'da $(i = x+1...)$ yap, rec'e (i)'yi ver.

• basta $x = -1$

Sepet Analizi

Association analysis → Veri madenciliğinin alanıdır bu.

1) kullanım analizi

$S \rightarrow A \rightarrow T$

Birbirini kullanıp matrisleri yapıp.

A içinde sayma ile T bul. T'nin max'ını

al. • T'izi sayarken de max bulabiliyorsun.

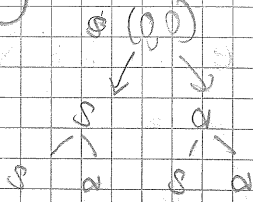
• Bunun kodlarına kesin bak.

1.27.00 kayda bak.

→ Ardışık en çok geçen kelimeler. (-k, K) vzağında en çok geçen kelimeler.



Lab-3



② For içinde yön ile kendini çağır.

① Gikmayı ayarlar.

13. hafta

Pointerlar Dizi Blemenlenme Erişim

* → dereferans yapar.

* Dizinin başında zaten bir pointer var.

* * (p & d) → * önceliklidir.

* (N scanf ile alınıp) int dizi[N]; → Illegal allocation
stack'li. Bu iş yapılırken yapılır. Ya stack'te yer yoksa?

* malloc (int size) → işletim sisteminden.

malloc (N * sizeof (int)); yer istiyor.

* "array" olarak pointer oluşturabilirsin. Bu pointer indexlenebilir.

* Bazen yer vermez. → if (array != null) de.

* Bölgeyle işin bittiyse alını

ide etmek lazım; free (array) → pointer alır.

* realloc () ve calloc () a bak.

→ Kummerbuz

→ Birthday Paradox

→ Görüntü işleme örneği

Beyaz noktaları gidermek için konvolüsyon yap.

* İlk "average filtre" yaptık.

Ama 0 ve 255'ler bir pürüz.

→ Bütün değerleri alırsan gürültü de dahil olur.

Çözüm → Medyan Filtresi] Ayrık noktaların sıyahları için de 1 tane beyaz gibi, elenin.

* Filtre boyutu arttıkça karışıklık artar.

→ Resm küçült $\textcircled{1}$ Alarak pixel al.

$\textcircled{2}$ 2×2 'lik ortalamayı pixel al.

→ Resm büyüt $\textcircled{1}$ İnterpolasyon yap.

büyütmek daha zor hangi tür interpol olduğu değişebilir.

Son hafta

• Paralel işlem süreyi kısaltır.

→ Matris çarpımı $\rightarrow T(n) = n^3$

* Strassen's matrix multiplication method $\rightarrow T(n) \cong n^{2.6}$

→ External (Merge) Sort

• Ram "volatile", elektrik gidiince ramde bir şey kalmaz.

HDD, SSD daha kalıcı ama daha yavaş.

RAM: 1GB

Data: 98B $= 9.2^{30} \cong 2^{33}$

Problem: Sort $\rightarrow 2^{66}$

* Bağlı liste, dizi gibi yapılar RAM'de oluşturulur.

CPU
1GHz
 10^9

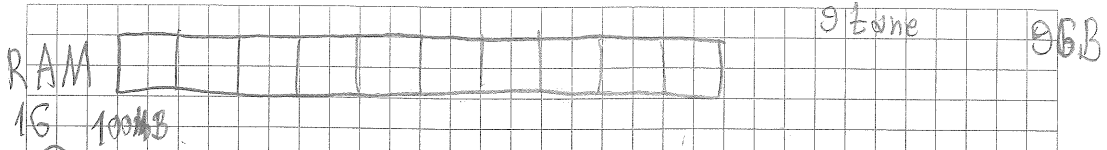
RAM
10mHz
 $10 \cdot 10^6$

100 kat daha hızlı

$\times 10^2$



HDD ^{1 1 1} ^{1 1}



- ① Verinin her bölümünü alıp tek tek sırala, geri koy.
- ② Her verinin ilk %10'unu RAM'in 9 yerine koy. 1 buffer
- ③ RAM bölümlerinin ilk %10'larını buffer'da sırala.
- ④ buffer dolunca HDD'ye yaz (başka bir alana). 9 bölgeden biri boşalınca alakalı sonraki %10'u al.

→ N sayı içinde toplam X olan kaç alt küme var.) N

→ Gezgin Satıcı (TSP) $(N-1)!$

* Sezgisel Algoritmalar: Çözüm uzayı çok büyük olduğunda bir kural/varsayım sınırlanır.

Örn: En yakın şehre git. $T(n) = n-1 + n-2 + \dots + 1 \approx n^2 \ll n!$

• Optimum çözümü hep karşılamaz.

Donald Knuth
Art of Comp Programming

* Genel kültürünü geliştir.

Robert Sedgewick
Algorithms

→ Genetik algoritmalar, Fast Fourier Transform

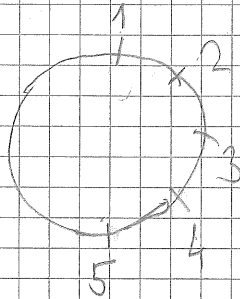
github
Clever Algorithms

Fin

- ① a) $N \cdot \log_2 N$
 b) $\log_2 N$
 c) 1
 d) N

1 3 9 24 48 49 80

- ② 5-3
 5-1
 5-2
 5-4



$N=5$
 $M=2$

④ $T(n) = 2^* T(n - \frac{1}{3}) + 1$

⑤ $T(n) = \frac{1}{a} \cdot T(n-b) + 1$

$a = \frac{1}{3}$

$T(n) = 3^{\log_2 n}$

$b = n/2$

⑥

1	2	3	M
0	0	1	
0	2	0	
3	0	0	

n

B - C - B

$M(i, j) =$

A

$\frac{1 \cdot 1}{1} \quad j=3$

$\frac{2 \cdot 2}{2}$

$\frac{3 \cdot 1}{3}$



$$\textcircled{7} \quad \begin{matrix} & & N \\ & & \hline (1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6) \end{matrix} \quad 15$$

$$2^{\text{da}} \Rightarrow \frac{N^2 - N}{2} = N^2 \quad 3^{\text{da}} \Rightarrow N^3$$

$$\textcircled{8} \quad \begin{matrix} N^4 \\ N^2 \\ i \cdot N \\ N \end{matrix} \quad \begin{matrix} N^2 \\ \sum_{i=1}^N \end{matrix}, \quad \begin{matrix} i \\ \sum_{j=1}^i \end{matrix} = \begin{matrix} N^2 \\ \sum_{i=1}^N \end{matrix}$$

$$\textcircled{9} \quad N \cdot M$$

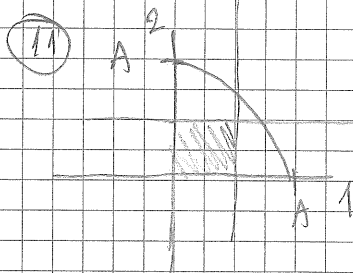
$$\textcircled{10} \quad \sum i = 0$$

$$\textcircled{1} \quad T = 1 \cdot N$$

$$\textcircled{2} \quad T = \frac{N}{2} \cdot (N+1)$$

$$\textcircled{3} \quad \frac{N(N+1)(N+2)}{4}$$

$$\frac{N(N+1)(N+2)}{2N}$$



$$x^2 + y^2 = A^2$$

$$S_1 = \frac{\pi r^2}{4} = \frac{\pi \cdot A^2}{4}$$

$$S_2 = \frac{A^2}{4}$$

$$13 - 1300$$

1, 4, 9, 16, 25, 36, 49, 64, 81,
100, 121, 144, 169, 196, 225, 256,
289, 324, 361, 400, 441, 484

$$a) < 500$$

$$① 500 \quad Y.$$

$$② x^2 \quad Y.$$

125, 216, 343

$$③ x^3 \quad D.$$

18, 27, 64

$$④ \frac{1\sqrt{\quad}}{1x}$$

$$b) > 500$$

529, 576, 625, 676, 729,
784, 841, 900, 961, 1024,
1089, 1156, 1225, 1296

542, 729, 1000

2 Sayı kaldı.

• < 500 ise

sayı b.kare.

Yalan-3 b.kare değil

ve

3. am küp ✓

sayı

> 500

kare x

küp ✓

sayı

< 500

kare

küp



(1/2)

$$\sum_{p=0}^N$$

$$\sum_{q=p+1}^{N-p} 1$$

$$\sum_{p=0}^N N-2p+1$$

0
1

$$N \rightarrow \frac{N(N+1)}{2}$$

$N-1$
 $N-2$

$$N-p-p+1$$

$$N-2p+1$$

13

MM
2 MM
Mh
M.N

5

20
30
60
80

9
8
2 6

Access
granted.



Yapısal Programlama

Yunus Emre
Selçuk

* Yapısal p. da Fonksiyonları uzun tutma. (10-15 satır)
Nesneye göre bu daha da kısılacak.

→ C'de kod yazımından bahsediyoruz.

uzunluğu

• main, 2 argüman olabilir: char dizisi ve int

• exit() → gives control back to os. Calling exit() in main() is same as using return: When main returns program exits.

• \n, \t, \r

* Makro nedir?

→ İşlemci

Komutları en başta yer alır, # ile başlar

It's capable of: 1) macro processing

2) conditional compiling 3) debugging with built-in macros

• # define

↳ gives descriptive names to nondescript numbers

↳ makes the code easier to change

→ Yerine değişken atayıp hafıza kullanmadan iş yaparsın.

• #include <~>

Berlirli özel bir
dizinde arar.

• #include "~~"

Kaynak kodun olduğu yere bakar.
Yoksa;



→ Introduced DevC to us

* To make com. pr. not close immediately:
system ("PAUSE");

→ Data Types

1) void

2) Scalar

3) Aggregate

arithmetic

pointer

enum

• Tümleyen alımları hatırladık

→ decimal %d, octal %o, hexadecimal %x

* Sayının başına 0 koyarsam "octal" algılanır.

0x → hexadecimal

* Read <limit>h

• scientific notation (%e)

↳ $3e2 = 300$ $5E1 = 50$

• 33,14159825... "%f", x → 33,14159825...

"%1.4f", x → 3,1415

Scapalar

global değişken

En başta

Harfler & /o.

• Program

→ static değişken

normal tanımlı

• File

→ static değişken

Main'in dışında en başta

• Function

→ go to

• Block

forlarda

```

int main () {
    fix Duration ();
    fix Duration ();
    return 0;
}

void fixDuration () {
    static int fx = 2;
    fx ++;
    printf ("%d", fx);
} // Önce 3, sonra 4 gelin.
// Var olanı için ikinci tanımı atlar.
    
```

Definition ve Allusion Global değişkeni başka dosyada

- All. için tanımıda kullanmak için gerekli.
- extern kelimesi kullanılır.
- Anlamı başka yerde tanımlı.

* Auto: gereksiz (For variables)

- static: fonksiyonda kalıcı, dosyada "file scope"
- extern: "Allusion" için
- const: sabit
- register: "Bu değişkeni register'a koy" → hızlandırır.
- Mesela string (register dışı) yaparsanız hep kullandığınız şeyi hızlandırır.

Memoryde değil fix yapamazsınız.
Adres yazmaz kullanmaz

- volatile: Derleyici değişkene yaptığı optimizasyonları durdurur. Mesela dış dosyadan değer okuyorsa her seferinde girilip çıkıp döner.

c0 = keyboard;

c1 = keyboard;

Kullanıcı dil için g.1. girer, kelime yazar.

...../...../..... Varsa da yoksa öneri.

Hi

4 hafta

II Her elemanı string olan matris.

```
#define MWORDS 5
      L LANG 2
```

strcpy

```
int main()
```

```
char *dizi [LANG][WORDS] =
```

```
{ {"abher", able, abarz, abarz, NULL}
```

```
{ absurde, accepter, accard, NULL } }
```

```
char str [10];
```

```
int diff = -1;
```

Her stringin başlangıç
adresini dizi'ye.

10. hafta

Hi

Self/Forward Referencing Structures

- Struktür alanında kendisi/başkası olmaz.
- Type'de'ye tanımlanma bunu yapamazsın.

→ Bağıli listeleri kendini referans eden struct'la yaparsın.

- Silme ve ekleme çok rahat.
- Dinamik bellekle kullanılır.

Natural Mismatch
Çözüm: char m1, m2;
short m3;

Alignment of Structure Mem. kene yap.

```
struct ALIGN {
```

```
char m1;
```

```
short m2;
```

```
char m3;
```

```
} st;
```

1000 1001 1002 1003

m1	m2	m3
----	----	----

1004 1005 1006 1007

m1	GAP	m2
m3		

Bazı derleyicilerde "int" i 4'ün
kübüne koyar gibi
karakterler dir.

...../...../.....

* Bellek çok önemliyse kullanılır.

Bosluk olmayacak şekilde tasarımı yapılır.

Bit Field

base type → name : length
int

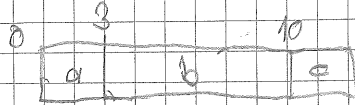
* BitField'ların
adresini almazsın.

struct {

int a:3;

int b:4;

int c:2;



Toplam boyut int'i (4 byte) kadar olabilir.

* Yani okurken sonuna &'la veremezsin.

Çözümü: Değeri değişkene al. Sonra bitField'ı atan.

UNION

Structure'a benzer.

typedef union {

struct {

char a1, a2;

} s;

long j;

float x;

} u;

* Union'daki her field ortak
aynı adresten başlar.

→ Elementlere bakarsın en büyüğü kadar
yer ayırır.

* Structure içinde olmadan

"int a:4" gibi şeyler yazamazsın.

→ Bu bir yandan alan tasarrufu sağlar.

→ Bazı işlemler önemli bitişime dir. (Intel)



* Mesela ödillerden puan hesaplayacakken. Puan dışında ödiller önemsiz. Vhish'a, isfine yazabilirsin.

Sayı & ise direkt yaz.

-1 $\rightarrow$ 11111111

0000001
11111110

Sayı - ise önce & yaz,
1'e göre 2: al ve 1 ekle.

12 $\rightarrow$ (00010001)₂

$$0101 + 0010 \rightarrow 0111$$

$$⑧ \quad 0101 + 1010 \rightarrow (1111)_2 = -1$$

$$1110 + 0011 \rightarrow -2 + 3 = 1$$

$$\begin{array}{r} -6 \\ 1010 \end{array} + \begin{array}{r} -2 \\ 1110 \end{array} \rightarrow -8$$

$$\begin{array}{r} 1110 \\ + 0011 \\ \hline 10001 \end{array} = 0001$$

$$(1000)_2$$

$$\begin{array}{r} 1001 \\ 0110 \end{array} \quad \begin{array}{r} 1101 \\ 0110 \end{array}$$

$$⑨ \quad 100.1010$$

bit	exponent	Mantissa
1	3	4

$$100.1010 \cdot 111 = 0.625$$

0 (nokta solunda)

$$\begin{array}{r} 0011.1001 \end{array}$$

$$\sim 1001 = .01001$$

$$3-4 = -1 \text{ (1br. } \leftarrow \text{)}$$

$$\frac{1}{4} + \frac{1}{32} = \frac{9}{32}$$

$$⑩ \quad 5.25 \rightarrow \underline{001.0} \quad (\text{Trunc. error})$$

Exp: 1010
 nokta 3 br sağa
 7-4=3 çıkmalı

Exp = 111
 En son gözükken yok
 $(0111 \ 1010)_2$

$$-4.375 \quad - (100.0111)$$

Sign	Exp.	Mant.
1	111	1000

$$\text{Exp} = 7 \quad 111$$

1 → 2 → 3

2 → 3 → 6

3 → 6 → 9

4 → 7 → 12

5 → 8 → 15

6 → 9 → 18

7 → 12 → 21

8 → 15 → 24

9 → 18 → 27

10 → 19 → 30

11 → 20 → 33

12 → 21 → 36

13 → 22 → 39

14 → 23 → 42

15 → 24 → 45

Ort

$$29 + 27 + 4 + 32 + 39 + 19 + 31$$

$$= 186$$

186

Sun

$$34 + 32 + 41 + 28 + 24 + 12 + 33$$

$$204$$

$$- 18$$

$$\frac{Shio}{9}$$

-1

Su

Sh

Ort

$$32$$

$$41$$

$$28$$

$$24$$

$$12$$

$$+ 33$$

$$170$$

↓

$$204$$

$$- 2 = 202$$

$$9$$

$$325 - 179$$

$$= 146$$

$$(PoE) 29$$

$$11$$

40

$$186$$

$$+ 18 = 202$$

→



①

1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23

* Bölün yoksa alması

2 - 6 - 10

* Alamyorsa hepini ver.

ekstra

②

$$f(x) = ax^2 + bx + c$$

$$f(f(x)) = 3x$$

$$f(ax^2 + bx + c) = a(ax^2 + bx + c)^2 + b(ax^2 + bx + c) + c$$

$$(ax^2 + bx + c)(ax^2 + bx + c)$$

$$c = 0$$

$$= a^2x^4 + 2abx^3 + 2acx^2 + b^2x + c^2$$

Köş tedi
Mehmet

$$\int \frac{dx}{\sqrt{4x-x^2}}$$

$$\int \frac{dx}{\sqrt{x} \sqrt{4-x}}$$

① Trig ② Alg, Diff ③ Krom,

$$\frac{A}{\sqrt{x}} + \frac{B}{\sqrt{4-x}} = \frac{1}{\sqrt{x}}$$

$$A = 2$$

$$A\sqrt{4-x} + B\sqrt{x} = 1$$

<u>k</u>	<u>basamak</u>	<u>sayi1</u>	<u>sayi2</u>	<u>sayi3</u>	<u>sayi4</u>	<u>Output</u>
2	0	0	1	1	-	
2	0	1	1	2	2	
2	1	1	1	2	0	
2	0	1	2	3	3	
2	1	1	2	3	0	
2	0	2	3	5	5	
2	1	2	3	5	0	
2	0	3	5	8	8	
2	1	3	5	8	0	
2	0	5	8	13	13	
2	1	5	8	13	1	
basamak=k ← 2	2	5	8	13	0	→ 13
2	0	8	13	21	21	
2	1	8	13	21	2	
2	2	8	13	21	0	→ 21
2	0	13	21	34	34	
2	1	13	21	34	3	
2	2	13	21	34	0	→ 34
2	0	21	34	55	55	
2	1	21	34	55	5	
2	2	21	34	55	0	→ 55
2	0	34	55	89	89	
2	1	34	55	89	8	
2	2	34	55	89	0	→ 89
2	0	55	89	144	144	
2	1	55	89	144	14	
2	2	55	89	144	1	
2	3	55	89	144	0	

Stop

k=2 ⇒ Output: 13, 21, 34, 55, 89

Algoritmanın Kısa Açıklaması

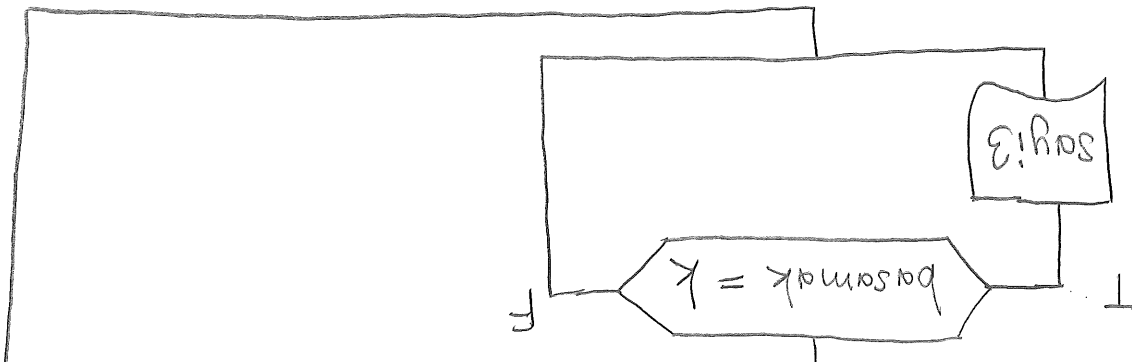
Önce k değeri, "basamak" ve "sayı" lar tanımlanır. $k=1$ durumu için tek basamaklı çıktılar yazdırılır.

Fibonacci sayı dizisinin ilk elemanları $sayi1$, $sayi2$ ve $sayi3$ 'e atanır. $sayi3$ sayısının basamak değeri k 'dan küçük olduğu sürece Fibonacci dizisinde ilerlenir.

	0	1	1	2	3	5	8	13	21	...
1. adım	$s1$	$s2$	$s3$							
2. adım		$s1$	$s2$	$s3$						
3. adım			$s1$	$s2$	$s3$					

Böylece dizideki sayılar tek tek $sayi3$ 'e atanır. İstenen basamak sayısı sağlanıyorsa ($\text{basamak} = k$) sayı yazdırılır.

$sayi3$ değeri $sayi4$ 'te saklanarak sürekli bölmelerle kaç basamaklı olduğu hesaplanır. Basamak sayısı k 'yı geçince ise döngü sona erer.



Start

'Basamak sayısı olan
k'yi giriniz.'

k

basamak = 0
sayi 1 = 0
sayi 2 = 1
sayi 3 = sayi 1 + sayi 2

F

k = 1

sayi 1

sayi 2

sayi 3

basamak ≤ k

Stop

basamak = 0
sayi 1 = sayi 2
sayi 2 = sayi 3
sayi 3 = sayi 1 + sayi 2
sayi 4 = sayi 3

sayi 4 > 0

sayi 4 = sayi 4 DIV 10
basamak = basamak + 1

	k	basamak	sayi1	sayi2	sayi3	sayi4	Output
(k=2 değeri için yapılan işlemler tekrar edilir.)	3	0	55	89	144	144	
	3	1	55	89	144	14	
	3	2	55	89	144	1	
	3	3	55	89	144	0	→ 144
	3	0	89	144	233	233	
	3	1	89	144	233	23	
	3	2	89	144	233	2	
	3	3	89	144	233	0	→ 233
	3	0	144	233	377	377	
	3	1	144	233	377	37	
	3	2	144	233	377	3	
	3	3	144	233	377	0	→ 377
	3	0	233	377	610	610	
	3	1	233	377	610	61	
	3	2	233	377	610	6	
	3	3	233	377	610	0	→ 610
basamak = k ←	3	0	377	610	987	987	
	3	1	377	610	987	98	
	3	2	377	610	987	9	
	3	3	377	610	987	0	→ 987
	3	0	610	987	1597	1597	
	3	1	610	987	1597	159	
	3	2	610	987	1597	15	
	3	3	610	987	1597	1	
basamak > k ←	3	4	610	987	1597	0	

Stop

k=3 ⇒ Output: 144, 233, 377, 610, 987

Analiz

	<u>k</u>	<u>basamak</u>	<u>sayi1</u>	<u>sayi2</u>	<u>sayi3</u>	<u>sayi4</u>	<u>Output</u>
	1	0	0	1	1	-	→ 0, 1, 1
	1	0	1	1	2	2	
basamak = k ←	1	1	1	1	2	0	→ 2
	1	0	1	2	3	3	
	1	1	1	2	3	0	→ 3
	1	0	2	3	5	5	
	1	1	2	3	5	0	→ 5
	1	0	3	5	8	8	
	1	1	3	5	8	0	→ 8
	1	0	5	8	13	13	
	1	1	5	8	13	1	
basamak > k ! ←	1	2	5	8	13	0	

k=1 ⇒ Output : 0, 1, 1, 2, 3, 5, 8